

Résumé de l'article

« XGBoost: A Scalable Tree Boosting System » — Tianqi Chen, Carlos Guestrin
(University of Washington), KDD 2016

Cet article présente XGBoost, un système open source de **gradient tree boosting** conçu pour être à la fois précis et extrêmement scalable. Il reprend les bases théoriques posées par Friedman (le gradient boosting de 2001) mais y ajoute une fonction objectif régularisée, un algorithme efficace de recherche de splits gérant la sparsité, et surtout une série d'optimisations systèmes (gestion du cache, compression, calcul out-of-core, parallélisation) qui permettent au système de traiter des milliards d'exemples avec des ressources limitées. Au moment de sa publication, XGBoost était utilisé dans 17 des 29 solutions gagnantes des compétitions Kaggle de 2015 et par toutes les équipes du top 10 du KDDCup 2015.

1. Fonction objectif régularisée

Un modèle d'ensemble d'arbres prédit la sortie comme la somme des scores de K arbres de régression (CART), chaque arbre associant à un exemple une feuille et un score continu. XGBoost introduit une fonction objectif régularisée $L(\phi) = \sum l(\hat{y}_i, y_i) + \sum \Omega(f_k)$, où l est une fonction de perte convexe différentiable et où le terme de régularisation $\Omega(f) = \gamma T + \frac{1}{2} \lambda \|w\|^2$ pénalise à la fois le nombre de feuilles T et l'amplitude des poids w de chaque feuille. Cette régularisation, inspirée des Regularized Greedy Forests mais plus simple et plus facile à paralléliser, vise à lisser les poids appris pour éviter le surapprentissage. Quand le paramètre de régularisation est nul, on retombe sur le gradient boosting classique.

2. Gradient tree boosting d'ordre 2

Comme le modèle ne peut pas être optimisé directement par les méthodes classiques (les arbres sont des fonctions, pas des paramètres dans un espace euclidien), l'apprentissage se fait de façon additive : à chaque itération t , on ajoute un arbre f_t qui minimise l'objectif. La contribution majeure de XGBoost par rapport au gradient boosting original de Friedman est d'utiliser une **approximation de Taylor du second ordre** de la perte (et non du premier ordre seulement), en s'appuyant sur le gradient g_i et la dérivée seconde (hessienne) h_i de la perte par rapport à la prédiction courante. Cette approximation quadratique permet, pour une structure d'arbre fixée, de calculer analytiquement le poids optimal de chaque feuille ($w_j^* = -\sum g_i / (\sum h_i + \lambda)$) ainsi qu'un score de qualité global de la structure de l'arbre, qui joue un rôle analogue à l'impureté de Gini dans les arbres de décision classiques, mais qui se généralise à n'importe quelle fonction de perte différentiable. Cette formule de score sert ensuite à évaluer le gain apporté par chaque split candidat, via une formule de réduction de perte qui combine les statistiques de gradient des sous-ensembles gauche et droit moins celles du nœud parent, le tout pénalisé par le coût de complexité γ .

3. Shrinkage et sous-échantillonnage des colonnes

Deux techniques complémentaires limitent le surapprentissage. La première, le **shrinkage** (introduit par Friedman), réduit la contribution de chaque nouvel arbre par un facteur d'apprentissage η , laissant de la place aux arbres suivants pour améliorer le modèle. La seconde est le **sous-échantillonnage des colonnes** (feature

subsampling), une technique empruntée aux forêts aléatoires : selon les retours d'utilisateurs, elle prévient le surapprentissage encore plus efficacement que le sous-échantillonnage des lignes, tout en accélérant les calculs parallèles.

4. Algorithmes de recherche de splits

L'**algorithme glouton exact** (exact greedy) énumère tous les splits possibles sur toutes les variables après avoir trié les données, ce qui garantit de trouver le meilleur split mais devient coûteux quand les données ne tiennent pas en mémoire. Pour pallier cela, XGBoost propose un **algorithme approché** qui propose des points de split candidats basés sur des percentiles de la distribution de chaque variable (en mode global, calculé une fois pour tout l'arbre, ou local, recalculé à chaque split), regroupe les données en buckets entre ces candidats, puis cherche le meilleur split parmi ces propositions seulement. Les expériences sur le jeu de données Higgs montrent que la version locale nécessite moins de candidats que la version globale pour atteindre la même précision, et que l'algorithme approché peut égaler l'algorithme exact avec un niveau d'approximation raisonnable.

Une contribution théorique importante est le **weighted quantile sketch** : pour proposer de bons points de split dans l'algorithme approché, il faut trouver des quantiles pondérés, où le poids de chaque exemple est sa dérivée seconde h_i (ce qui correspond à reformuler l'objectif comme une perte quadratique pondérée). Aucun algorithme existant ne savait calculer des quantiles sur des données pondérées avec une garantie théorique ; les auteurs en proposent un nouveau, distribuable, basé sur une structure de données supportant des opérations de fusion (merge) et d'élagage (prune), chacune préservant une borne d'erreur prouvée ϵ . La preuve complète de cette structure est détaillée dans l'annexe de l'article.

5. Algorithme conscient de la sparsité (sparsity-aware)

Les données réelles sont souvent creuses (valeurs manquantes, encodage one-hot, entrées nulles fréquentes). XGBoost introduit une **direction par défaut** apprise pour chaque nœud de l'arbre : quand une valeur est manquante, l'exemple est automatiquement dirigé vers la branche par défaut, dont la direction optimale est apprise à partir des données plutôt que fixée arbitrairement. L'algorithme ne visite que les entrées non manquantes, ce qui rend le coût de calcul linéaire en nombre d'entrées présentes plutôt qu'en nombre total d'entrées. Sur le jeu de données Allstate-10K (creux à cause d'un encodage one-hot), cette approche s'avère environ 50 fois plus rapide qu'une implémentation naïve ignorant la sparsité.

6. Conception du système

Plusieurs innovations purement systèmes expliquent la scalabilité de XGBoost. La structure de **block en colonnes** (column block) stocke les données triées par variable dans un format compressé en colonnes (CSC), calculé une seule fois puis réutilisé à chaque itération, ce qui transforme la recherche de quantiles en un simple parcours linéaire et permet de paralléliser la collecte des statistiques par colonne. L'analyse de complexité montre que cette structure permet d'économiser un facteur logarithmique ($\log n$) par rapport à l'algorithme naïf.

L'**accès conscient du cache** (cache-aware access) corrige un problème pratique : les accès indirects aux statistiques de gradient par index de ligne provoquent des défauts de cache lorsque les données sont grandes. XGBoost résout cela par préchargement (prefetching) dans des buffers internes par thread pour l'algorithme exact, et par un choix optimal de la taille des blocks pour l'algorithme approché (ni trop petit, pour garder une bonne parallélisation, ni trop grand, pour éviter les défauts de cache). Les expériences montrent un gain de vitesse d'un facteur deux sur de grands jeux de données.

Enfin, le **calcul out-of-core** permet de traiter des données ne tenant pas en mémoire en les divisant en blocks stockés sur disque, préchargés par un thread indépendant pendant que le calcul se poursuit. Deux techniques améliorent encore le débit disque : la **compression par block** (compression des valeurs de variables et codage compact des index de ligne, pour un taux de compression d'environ 26 à 29 %) et le **sharding** des données sur plusieurs disques en parallèle. Ensemble, ces techniques permettent de traiter 1,7 milliard d'exemples sur une seule machine.

7. Évaluations expérimentales

Les auteurs comparent XGBoost à scikit-learn, R gbm, pGBRT, Spark MLlib et H2O sur quatre jeux de données : Allstate (assurance, 10M lignes, très creux), Higgs Boson (physique des particules, 10M lignes), Yahoo LTRC (apprentissage du ranking) et Criteo (1,7 milliard de lignes, click-through rate). Sur Higgs-1M, XGBoost atteint une précision (AUC) comparable à scikit-learn mais environ 10 fois plus rapide, tout en étant bien plus précis que R gbm. Sur Yahoo LTRC, XGBoost dépasse pGBRT en vitesse, et le sous-échantillonnage de colonnes y améliore même légèrement la métrique NDCG@10 en réduisant le surapprentissage. Dans le cadre out-of-core sur les données Criteo, la compression accélère les calculs d'un facteur 3 et le sharding sur deux disques d'un facteur 2 supplémentaire. Enfin, en environnement distribué sur un cluster YARN de 32 machines EC2, XGBoost s'exécute plus de 10 fois plus vite que Spark MLlib par itération et 2,2 fois plus vite que la version optimisée de H2O, tout en étant le seul système capable de traiter l'intégralité des 1,7 milliard d'exemples grâce au calcul out-of-core ; sa performance reste par ailleurs quasi linéaire lorsqu'on augmente le nombre de machines.

Conclusion

XGBoost démontre qu'un gain de performance majeur en machine learning ne provient pas uniquement d'innovations algorithmiques (objectif régularisé d'ordre 2, weighted quantile sketch, gestion de la sparsité) mais aussi, et peut-être surtout, d'une conception système soignée : structures de données adaptées au cache processeur, compression, parallélisation et calcul out-of-core. C'est la combinaison de ces deux dimensions — algorithmique et systèmes — en un système de bout en bout qui permet à XGBoost de traiter des problèmes à l'échelle du milliard d'exemples avec un minimum de ressources de calcul, et qui explique son adoption massive comme outil de référence en data science.